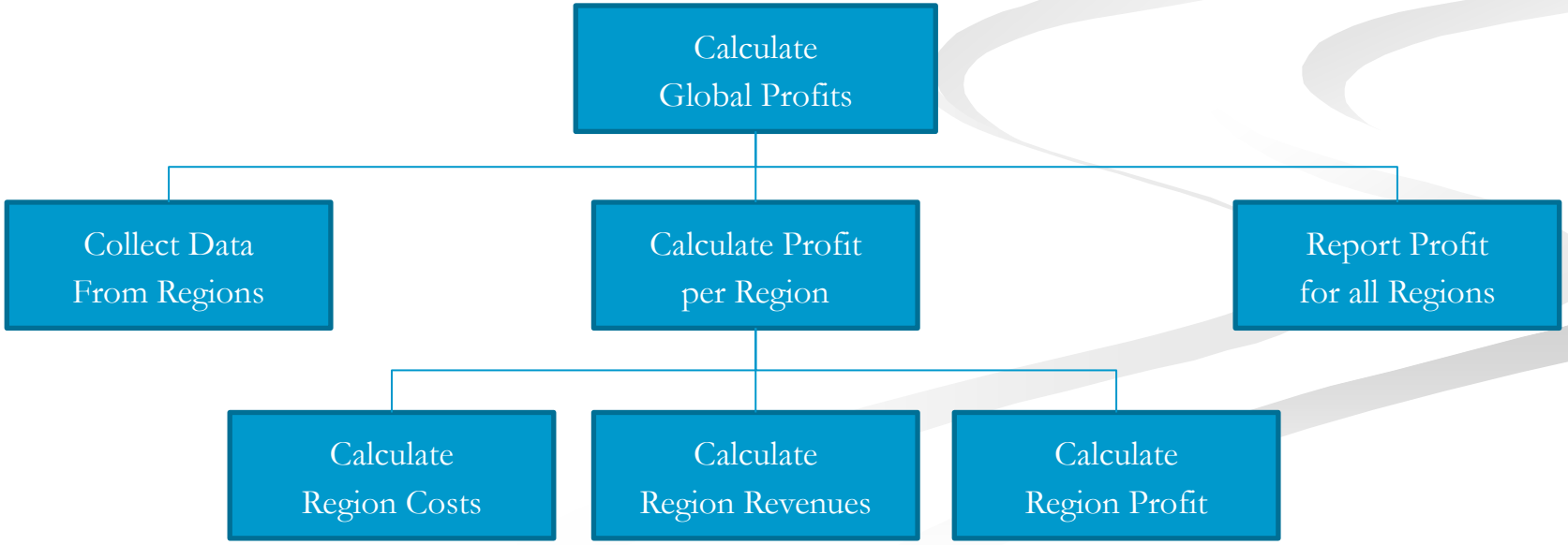
Structured Problem Solving (SPS)

Implementation Model

Pseudocode

Structure of Models Used in Problem Solving
Programming Example hierarchy and modularity

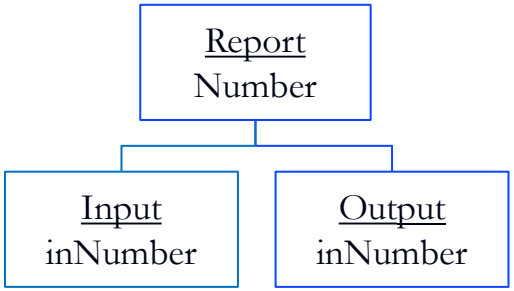
- **Hierarchy** decomposes tasks and processes into **modules**
- In programming these modules can be made into functions
- This is the blueprint to start the pseudocode



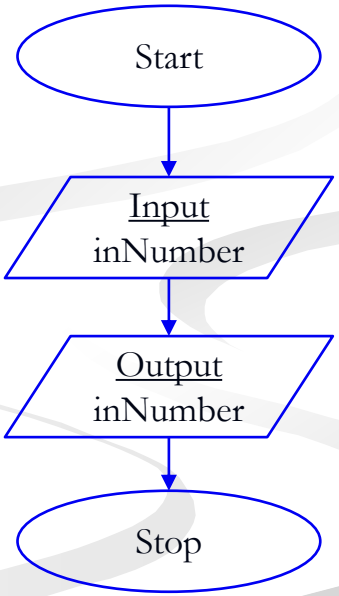
Implementation Model

Hierarchy chart – Pseudocode - Flowchart

Problem Statement:
"Develop a program that allows a user to enter a number, and that number is immediately displayed after it is entered."



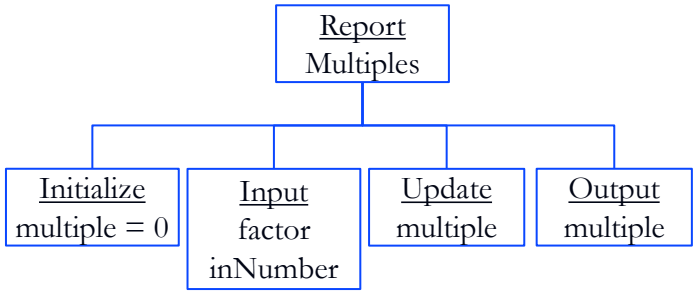

```
main()
{
   input inNumber
   output inNumber
   stop
}  # end main
```



Implementation Model

Pseudocode single number and multiple

Problem Statement:
"Develop a program that allows a user to enter a multiplicative factor and a number. The number multiplied by the factor is displayed immediately after the number is entered."



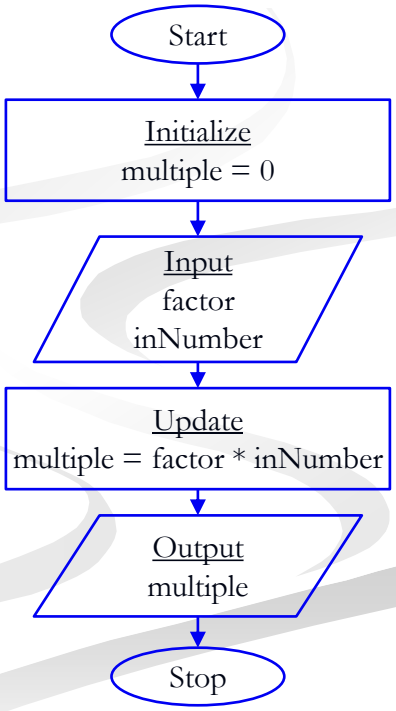

```
main()
{
    call initVars()
    call getInputs()
    call updateMult()
    call outputMult()
    stop
}  # end main

initVars()
{ multiple = 0 }  # end initVars

getInputs()
{
    input factor
    input inNumber
} # end getInputs

updateMult()
{
    multiple = factor * inNumber
} # end updateMult

outputMult ()
{ output multiple }  # end outputMult
```



Implementation Model

Fixed number of inputs and outputs

Problem Statement:

"Develop a program that allows a user to enter a multiplicative factor, and then to enter 4 numbers, one at a time. Each number multiplied by the factor is displayed immediately after the number is entered. After 4 numbers have been entered, the program ends."

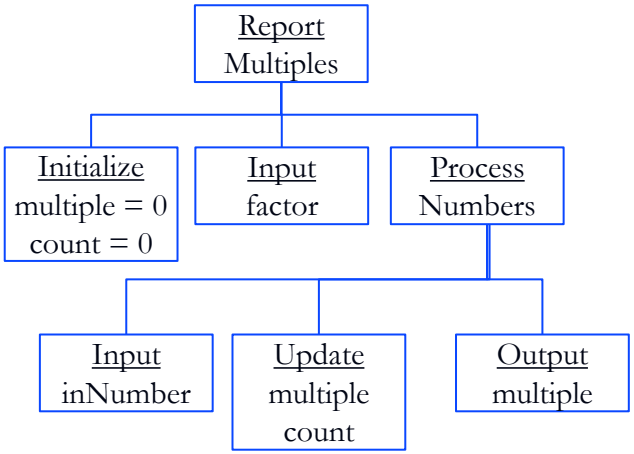
I/O Design:

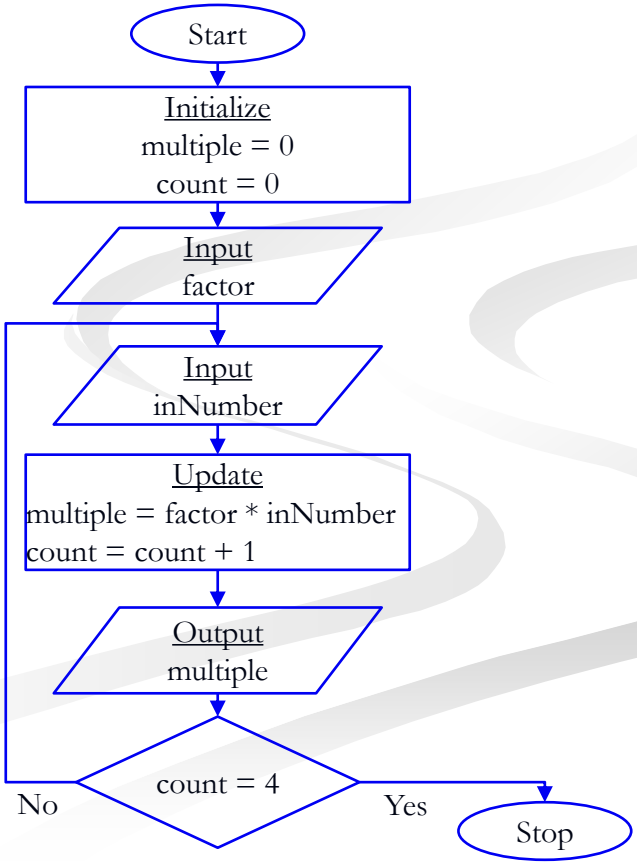
Output:

- multiple, (number multiplied by factor)

Input:

- factor, (number entered as a multiplier)
- inNumber, (number entered)

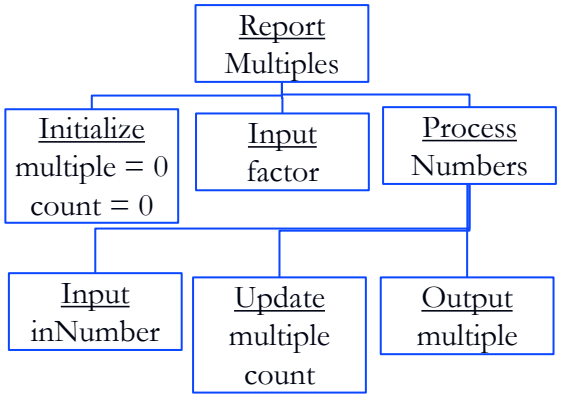


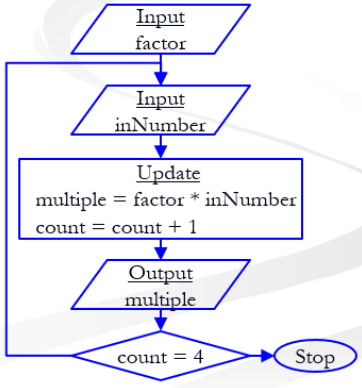


Implementation Model

Pseudocode fixed number of inputs and outputs

Problem Statement:
"Develop a program that allows a user to enter a multiplicative factor, and then to enter 4 numbers, one at a time. Each number multiplied by the factor is displayed immediately after the number is entered. After 4 numbers have been entered, the program ends."






```
main()
{
    call initVars()
    call getFactor()
    while (count < 4) {
        call processNums()
    }
    stop
}  # end main
```

```
initVars()
{
    multiple = 0
    count = 0
}  # end initVars

getFactor()
{
    input factor
} # end getFactor

processNums ()
{
    input inNumber
    call updateVars()
    output multiple
}  # end processNums

updateVars()
{
    multiple = factor * inNumber
    count = count + 1
} # end updateVars
```

Implementation Model

Inputs and outputs with a sentinel

Problem Statement:

"Develop a program that allows a user to enter a multiplicative factor, and then to enter numbers, one at a time. Each number multiplied by the factor is displayed immediately after the number is entered. The user enters numbers until the sentinel value of 999 is entered, and then the program ends."

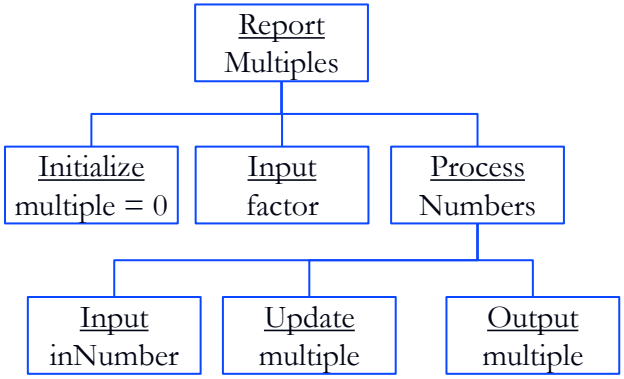
I/O Design:

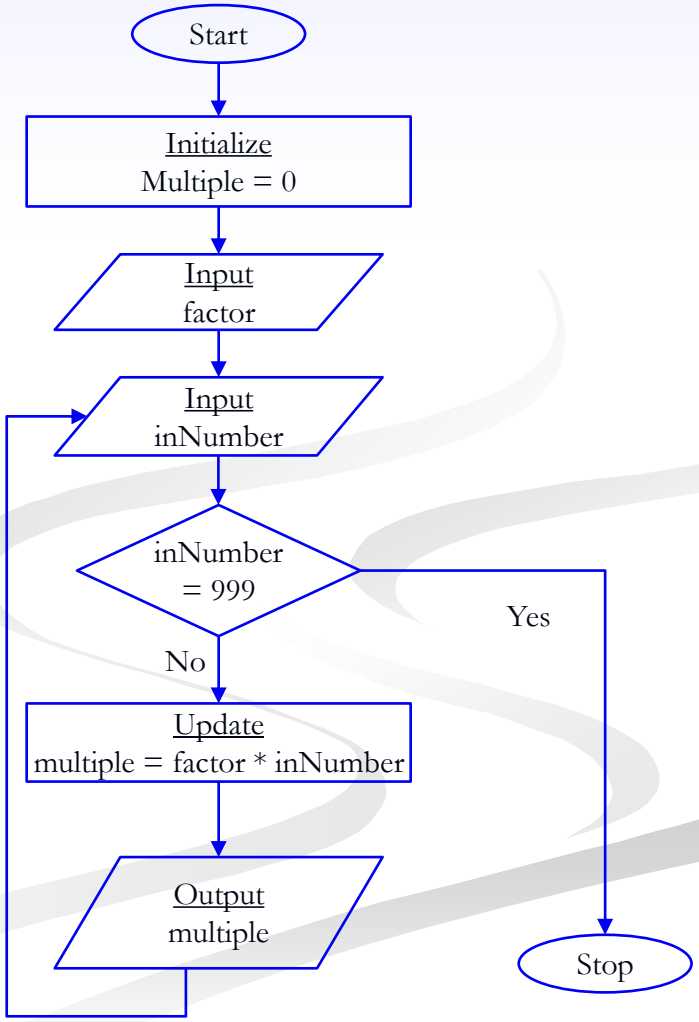
Output:

- multiple, (number multiplied by factor)

Input:

- factor, (number entered as a multiplier)
- inNumber, (number entered)



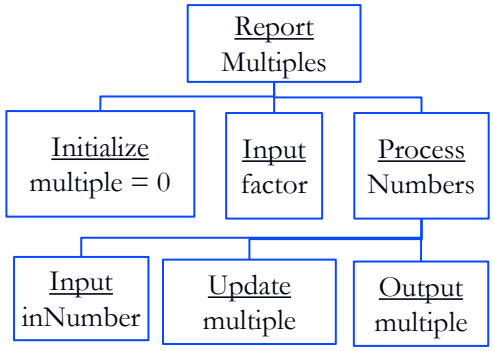


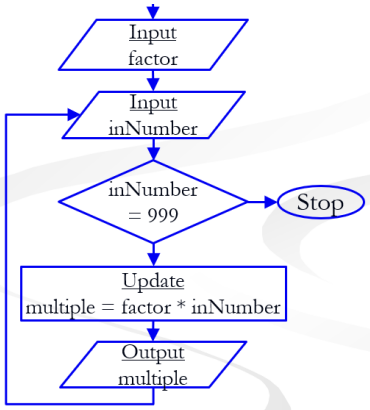
Implementation Model

Pseudocode – inputs outputs and sentinel

Problem Statement:

"Develop a program that allows a user to enter a multiplicative factor, and then to enter numbers, one at a time. Each number multiplied by the factor is displayed immediately after the number is entered. The user enters numbers until the sentinel value of 999 is entered, and then the program ends."






```
main()
{
   call initVars()
   call getFactor()
   call getInput()
   while (inNumber != 999) {
      call processNums()
   }
   stop
}  # end main
```

```
initVars()
{
   multiple = 0
}  # end initVars

getFactor()
{
   input factor
} # end getFactor

getInput()
{
   input inNumber
} # end getInput

processNums ()
{
   multiple = factor * inNumber
   output multiple
   call getInput()
}  # end processNums
```

Implementation Model

Cardinality, multiples, and sums

Problem Statement:

"Develop a program that allows a user to enter a multiplicative factor and a count of numbers that will be entered. The user then enters numbers, one at a time until the count is reached. The number multiplied by the factor is displayed immediately after the number is entered. After all numbers have been entered, the program displays separately the sum of all the numbers and the sum of all the multiples."

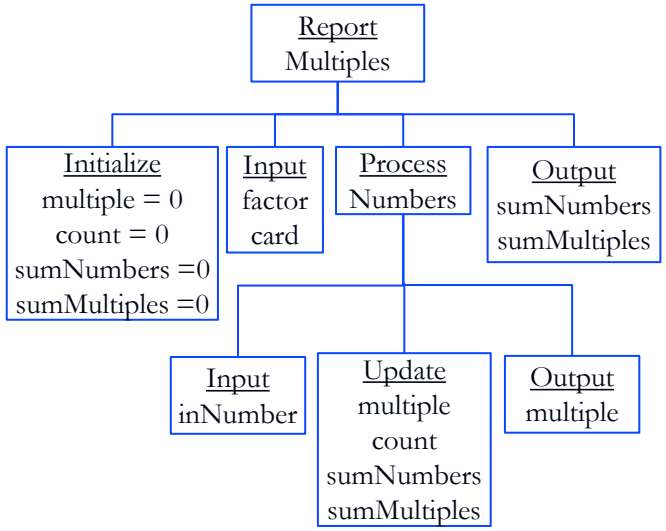
I/O Design:

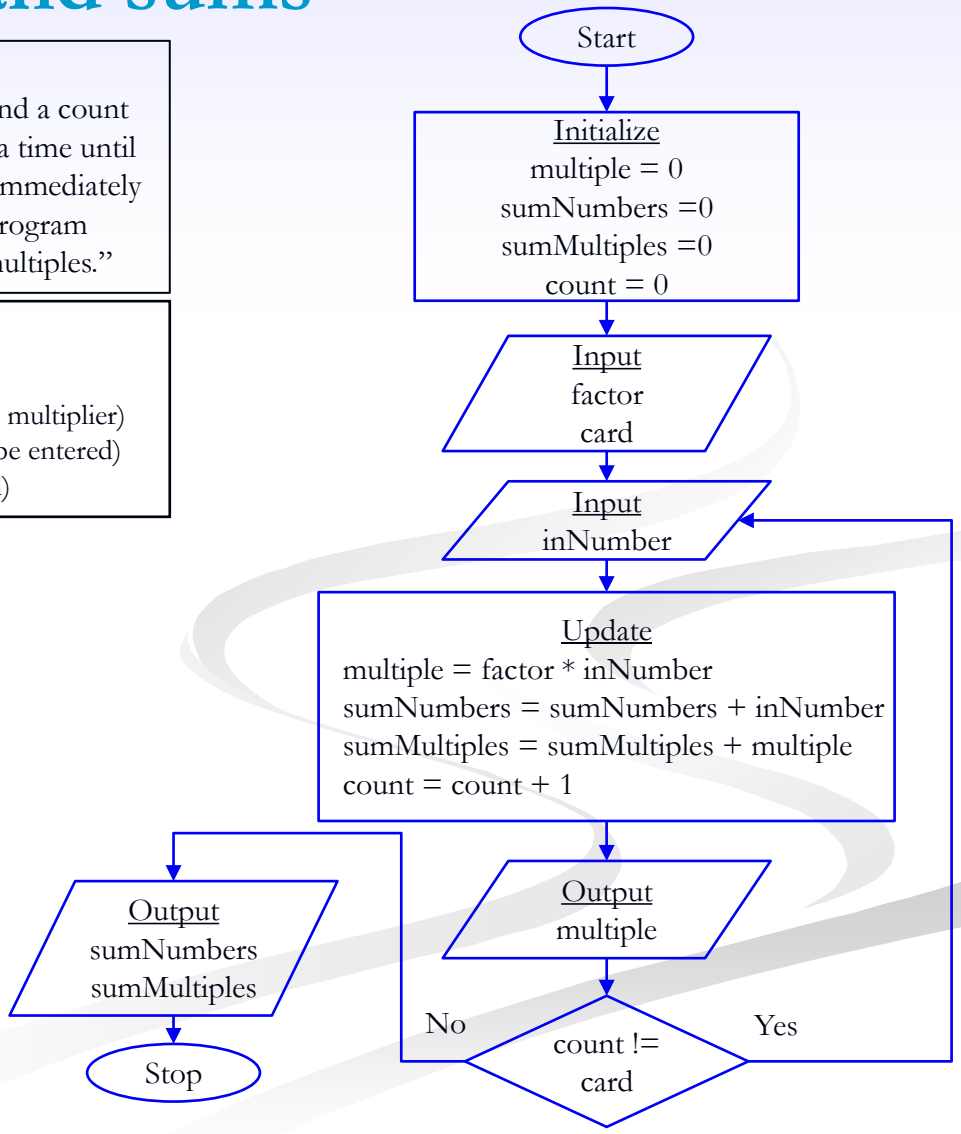
Output:

- multiple, (number times the factor)
- sumNumbers, (sum of numbers entered)
- sumMultiples, (sum of multiples)

Input:

- factor, (number entered as a multiplier)
- card, (count of numbers to be entered)
- inNumber, (number entered)



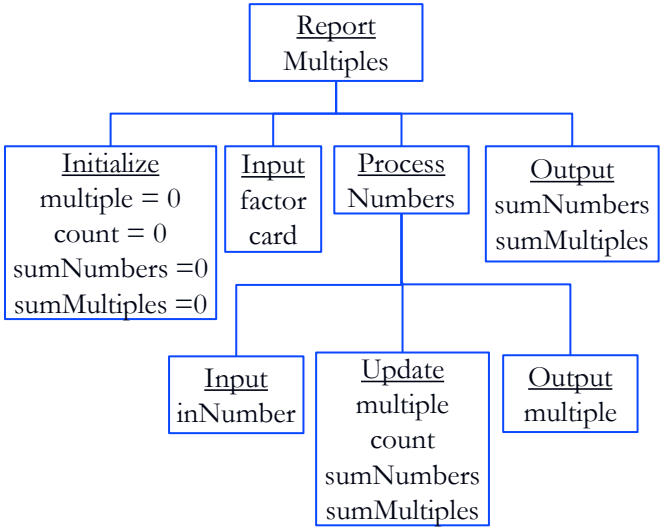


Implementation Model

Pseudocode – Cardinality, multiples, sums

Problem Statement:

"Develop a program that allows a user to enter a multiplicative factor and a count of numbers that will be entered. The user then enters numbers, one at a time until the count is reached. The number multiplied by the factor is displayed immediately after the number is entered. After all numbers have been entered, the program displays separately the sum of all the numbers and the sum of all the multiples."




```
main()
{
    call initVars()
    call getInputs()
    while (count != card) {
        call processNums()
    }
    call outputSummaryStats()
    stop
}  # end main
```

```
initVars()
{
    multiple = count = 0
    sumNumbers = sumMultiples = 0
}  # end initVars

getInputs()
{
    input factor, input card
}  # end getInputs

processNums ()
{
    input inNumber
    multiple = factor * inNumber
    sumNumbers = sumNumbers + inNumber
    sumMultiples = sumMultiples + multiple
    count = count + 1
    output multiple
}  # end processNums

outputSummaryStats ()
{
    output sumNumbers
    output sumMultiples
}  # end outputSummaryStats
```